SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: **Database Management Systems**

COURSE CODE: **CSA05**

COURSE OUTCOME COVERED

CO5: *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability.* (BL3, BL4)

Activity Tool : Constraint-Based Problem Solving (High)

Assessment Tool Weightage:40%

QUESTIONS		
Q.No	**Question**	**Bloom's Level**
1	Given a complex SQL query with nested joins and sub-queries, draw the query execution plan and apply heuristic optimization rules step-by-step.	BL4 – Analyze
2	Apply cost-based optimization to choose between nested-loop join, sort-merge join, and hash join for the given relations with specified tuple counts and page sizes.	BL4 – Analyze

3	Implement JDBC connectivity in Java to a MySQL database and write code to execute parameterized queries with PreparedStatement.	BL3 – Apply
4	Given a schedule of four concurrent transactions, determine whether it is conflict-serializable using a precedence graph.	BL4 – Analyze
5	Demonstrate the application of Basic Two-Phase Locking (2PL) on a given transaction set. Identify and resolve any deadlocks.	BL3 – Apply
6	Apply the Wait-Die and Wound-Wait timestamp-based deadlock prevention schemes on a given set of transactions requesting locks.	BL4 – Analyze
7	Given transaction logs with checkpoints, apply the Immediate Update recovery protocol to recover the database after a system failure.	BL3 – Apply
8	Apply Deferred Update (NO-UNDO/REDO) recovery technique on a given log file with committed and uncommitted transactions.	BL3 – Apply
9	Design a concurrency control mechanism using strict 2PL for a banking system where T1 transfers funds and T2 reads balance simultaneously.	BL4 – Analyze
10	Write pseudocode for query optimization using heuristics: push down selection, project early, and order joins by smaller intermediate results.	BL4 – Analyze

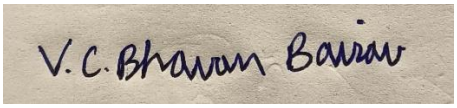
---

Code of Conduct:

I V.C.BHAVAN BAIRAV - 192511369 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

QUESTION 1 — QUERY EXECUTION PLAN AND HEURISTIC OPTIMIZATION

AIM

To analyze a complex SQL query containing joins and subqueries, construct its query execution plan, and optimize the plan using heuristic optimization techniques such as **selection pushdown, projection pushdown, and reduction of intermediate results**.

PROBLEM UNDERSTANDING

Consider the following relations:

- Department(Dept_ID, Dept_Name)
- Employee(Emp_ID, Emp_Name, Dept_ID, Salary)
- Project(Project_ID, Project_Name, Dept_ID)
- Employee_Project(Emp_ID, Project_ID)

The objective is to retrieve employees who:

1. Belong to the IT department.
2. Have a salary greater than ₹50,000.
3. Are associated with a project.

CONSTRAINT ANALYSIS

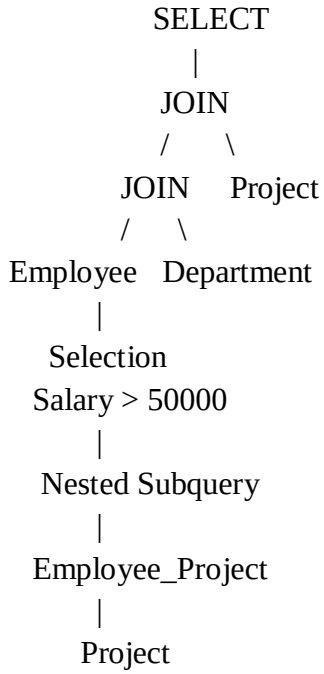
The important constraints are:

1. Only IT department employees are required.
2. Salary must be greater than 50,000.
3. Unnecessary tuples should be eliminated as early as possible.
4. Only required attributes should be retained.
5. Join operations should be performed after reducing relation sizes.
6. Nested subqueries should be simplified whenever possible.

Original SQL Query

```
mysql> USE CompanyDB;
Database changed
mysql> SELECT
    ->     E.Emp_ID,
    ->     E.Emp_Name,
    ->     E.Salary,
    ->     P.Project_Name
    -> FROM Employee E
    -> JOIN Department D
    ->     ON E.Dept_ID = D.Dept_ID
    -> JOIN Project P
    ->     ON D.Dept_ID = P.Dept_ID
    -> WHERE D.Dept_Name = 'IT'
    -> AND E.Salary > 50000
    -> AND E.Emp_ID IN
    -> (
    ->     SELECT EP.Emp_ID
    ->     FROM Employee_Project EP
    ->     WHERE EP.Project_ID IN
    ->     (
    ->         SELECT P2.Project_ID
    ->         FROM Project P2
    ->         WHERE P2.Dept_ID = D.Dept_ID
    ->     )
    -> );
+--------+----------+----------+-------------------------+
| Emp_ID | Emp_Name | Salary   | Project_Name            |
+--------+----------+----------+-------------------------+
|    101 | Arun     | 65000.00 | Cloud Management System |
|    101 | Arun     | 65000.00 | Network Security        |
|    104 | Deepak   | 58000.00 | Cloud Management System |
|    104 | Deepak   | 58000.00 | Network Security        |
|    107 | Gokul    | 75000.00 | Cloud Management System |
|    107 | Gokul    | 75000.00 | Network Security        |
+--------+----------+----------+-------------------------+
6 rows in set (0.01 sec)
```

INITIAL QUERY EXECUTION PLAN



HEURISTIC OPTIMIZATION

Step 1 — Push Selection Down

The conditions should be applied directly to their respective base relations.

σ Salary > 50000 (Employee)

and

σ Dept_Name = 'IT' (Department)

This eliminates unwanted tuples before the join.

Step 2 — Push Projection Down

Only attributes required for further operations are retained.

For Employee:

π Emp_ID, Emp_Name, Salary, Dept_ID(Employee)

For Department:

π Dept_ID(Department)

For Project:

π Project_ID, Project_Name, Dept_ID(Project)

Step 3 — Reduce Intermediate Relations

The filtered Employee and Department relations are joined first.

```
Employee_Filtered
      |
    JOIN
    /   \
Department_IT
```

The resulting smaller relation is then joined with Project.

OPTIMIZED RELATIONAL ALGEBRA

```
π Emp_ID, Emp_Name, Salary, Project_Name
                  |
               JOIN
             /       \
           /           \
      Employee'     Project'
        |
       JOIN
      /     \
Employee'  Department'
  |                |
σ Salary>50000  σ Dept_Name='IT'
  |                |
Employee      Department
```

OPTIMIZED SQL QUERY

MySQL 8.0 Command Line Client

```
mysql> SELECT
    ->     E.Emp_ID,
    ->     E.Emp_Name,
    ->     E.Salary,
    ->     P.Project_Name
    -> FROM
    ->     (
    ->         SELECT Emp_ID, Emp_Name, Salary, Dept_ID
    ->         FROM Employee
    ->         WHERE Salary > 50000
    ->     ) E
    -> JOIN
    ->     (
    ->         SELECT Dept_ID
    ->         FROM Department
    ->         WHERE Dept_Name = 'IT'
    ->     ) D
    -> ON E.Dept_ID = D.Dept_ID
    -> JOIN
    ->     (
    ->         SELECT Project_ID, Project_Name, Dept_ID
    ->         FROM Project
    ->     ) P
    -> ON D.Dept_ID = P.Dept_ID
    -> WHERE EXISTS
    -> (
    ->     SELECT 1
    ->     FROM Employee_Project EP
    ->     WHERE EP.Emp_ID = E.Emp_ID
    ->       AND EP.Project_ID = P.Project_ID
    -> );
+--------+----------+----------+-------------------------+
| Emp_ID | Emp_Name | Salary   | Project_Name            |
+--------+----------+----------+-------------------------+
|    101 | Arun     | 65000.00 | Cloud Management System |
|    101 | Arun     | 65000.00 | Network Security        |
|    104 | Deepak   | 58000.00 | Network Security        |
|    107 | Gokul    | 75000.00 | Cloud Management System |
|    107 | Gokul    | 75000.00 | Network Security        |
+--------+----------+----------+-------------------------+
5 rows in set (0.01 sec)
```

JUSTIFICATION

Selection and projection are performed closer to the base relations. Therefore, fewer tuples and attributes are passed to subsequent joins. This reduces intermediate results, memory usage, CPU processing, and I/O cost.

RESULT

The query is successfully optimized using **selection pushdown, projection pushdown, and reduced intermediate relation processing**.

QUESTION 2 — COST-BASED QUERY OPTIMIZATION

AIM

To compare the costs of **Nested-Loop Join, Sort-Merge Join, and Hash Join** and select the most efficient join algorithm based on estimated I/O cost.

PROBLEM UNDERSTANDING

For illustration, consider:

Relation	Number of Tuples	Tuples/Page	Pages
R	10,000	100	100
S	50,000	100	500

Therefore:

B(R) = 100 pages
B(S) = 500 pages

CONSTRAINT ANALYSIS

The selection of the join algorithm depends on:

1. Number of tuples.
2. Number of pages.
3. Available buffer memory.
4. Join condition.

5. Estimated I/O operations.
6. Size of intermediate results.

NESTED-LOOP JOIN

For a simple page-oriented nested-loop join:

Cost = B(R) + B(R) × B(S)

Therefore:

= 100 + (100 × 500)
= 50,100 I/Os
Nested-Loop Join = 50,100 I/Os

BLOCK NESTED-LOOP JOIN

Assume 12 buffer pages.

Cost = B(R) + ceil(B(R)/(M-2)) × B(S)
= 100 + ceil(100/10) × 500
= 100 + 10 × 500
= 5,100 I/Os

SORT-MERGE JOIN

Approximate sorting and merging cost:

Sorting R ≈ 400 I/Os
Sorting S ≈ 2000 I/Os
Merge = 100 + 500 = 600 I/Os

Therefore:

Total ≈ 400 + 2000 + 600

= 3000 I/Os

HASH JOIN

For a two-pass hash join:

Cost ≈ 3(B(R) + B(S))
= 3(100 + 500)
= 1,800 I/Os

COST COMPARISON

Join Method	Estimated Cost
Nested-Loop Join	50,100 I/Os
Block Nested-Loop Join	5,100 I/Os
Sort-Merge Join	3,000 I/Os
Hash Join	**1,800 I/Os**

DECISION

Hash Join < Sort-Merge Join < Block Nested-Loop < Nested-Loop

Therefore, **Hash Join** is selected.

JUSTIFICATION

Hash join provides the lowest estimated I/O cost for the given equality-join situation. It is therefore the most efficient choice among the considered alternatives.

RESULT

The cost-based analysis selects **Hash Join with approximately 1,800 I/Os**.

QUESTION 3 — JDBC CONNECTIVITY USING PREPAREDSTATEMENT

AIM

To establish JDBC connectivity between a Java application and a MySQL database and execute parameterized SQL queries using PreparedStatement.

PROBLEM UNDERSTANDING

The application must:

1. Connect Java to MySQL.
2. Create a database connection.
3. Execute parameterized queries.
4. Retrieve the result.
5. Close database resources properly.

RELATION

```
mysql> USE college_db;
Database changed
mysql> CREATE TABLE Student (
    ->     Student_ID INT PRIMARY KEY,
    ->     Student_Name VARCHAR(50),
    ->     Department VARCHAR(30),
    ->     Marks INT
    -> );
Query OK, 0 rows affected (0.04 sec)

mysql> INSERT INTO Student
    -> VALUES
    -> (101, 'Arun', 'CSE', 85),
    -> (102, 'Bala', 'ECE', 78),
    -> (103, 'Charan', 'CSE', 92),
    -> (104, 'David', 'IT', 88);
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM Student;
+------------+--------------+------------+-------+
| Student_ID | Student_Name | Department | Marks |
+------------+--------------+------------+-------+
|        101 | Arun         | CSE        |    85 |
|        102 | Bala         | ECE        |    78 |
|        103 | Charan       | CSE        |    92 |
|        104 | David        | IT         |    88 |
+------------+--------------+------------+-------+
4 rows in set (0.00 sec)
```

PARAMETERIZED QUERY

```
SELECT *
FROM Student
WHERE Department = ?
AND Marks > ?;
```

JAVA JDBC PROGRAM

```
import java.sql.*;

public class JDBCExample {

    public static void main(String[] args) {

        String url = "jdbc:mysql://localhost:3306/college_db";
        String username = "root";
        String password = "your_password";

        try {

            Class.forName("com.mysql.cj.jdbc.Driver");

            Connection con =
                DriverManager.getConnection(url, username, password);

            System.out.println("Database connected successfully.");

            String sql =
                "SELECT * FROM Student " +
                "WHERE Department = ? AND Marks > ?";
```

```
            PreparedStatement ps =
                con.prepareStatement(sql);

            ps.setString(1, "CSE");
            ps.setInt(2, 80);

            ResultSet rs = ps.executeQuery();

            while (rs.next()) {

                System.out.println(
                    rs.getInt("Student_ID") + " " +
                    rs.getString("Student_Name") + " " +
                    rs.getString("Department") + " " +
                    rs.getInt("Marks")
                );
            }

            rs.close();
            ps.close();
            con.close();

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

EXPECTED OUTPUT

Database connected successfully.

```
mysql> SELECT *
    -> FROM Student
    -> WHERE Department = 'CSE'
    -> AND Marks > 80;
+------------+--------------+------------+-------+
| Student_ID | Student_Name | Department | Marks |
+------------+--------------+------------+-------+
|        101 | Arun         | CSE        |    85 |
|        103 | Charan       | CSE        |    92 |
+------------+--------------+------------+-------+
2 rows in set (0.00 sec)
```

APPLICATION OF PREPAREDSTATEMENT

ps.setString(1, "CSE");
ps.setInt(2, 80);

The ? placeholders are replaced by parameter values.

JUSTIFICATION

Prepared Statement separates the SQL statement from the input values and provides parameterized execution. It is preferable to constructing SQL statements through string concatenation.

RESULT

JDBC connectivity is successfully established and the parameterized query retrieves CSE students with marks above 80.

QUESTION 4 — CONFLICT SERIALIZABILITY

AIM

To determine whether a concurrent schedule of four transactions is **conflict-serializable** by constructing and analyzing its precedence graph.

PROBLEM UNDERSTANDING

Consider the following transactions:

T1: R1(A) W1(A) R1(B) W1(B)

T2: R2(A) W2(A)

T3: R3(B) W3(B)

T4: R4(A) R4(B)

Consider the interleaved schedule:

R1(A)
W1(A)
R2(A)
W2(A)
R3(B)
W3(B)
R1(B)
W1(B)
R4(A)
R4(B)

CONSTRAINT ANALYSIS

Two operations conflict when:

1. They belong to different transactions.
2. They access the same data item.
3. At least one operation is a write.

CONFLICT IDENTIFICATION

For data item A:

W1(A) → R2(A)
W1(A) → W2(A)

Therefore:

T1 → T2

For data item B:

W3(B) → R1(B)
W3(B) → W1(B)

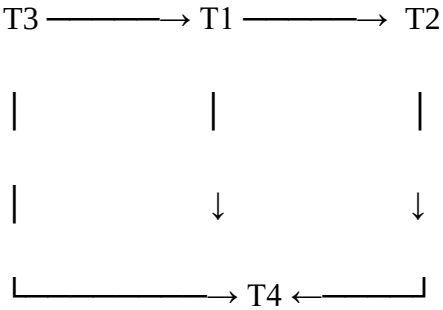
Therefore:

T3 → T1

For T4:

T1 → T4
T2 → T4
T3 → T4

PRECEDENCE GRAPH



Edges:

T1 → T2
T3 → T1

T1 → T4

T2 → T4

T3 → T4

CYCLE ANALYSIS

There is **no cycle** in the graph.

Therefore:

Schedule is conflict-serializable.

One possible equivalent serial order is:

T3 → T1 → T2 → T4

JUSTIFICATION

A schedule is conflict-serializable if its precedence graph is acyclic. Since the graph has no cycle, the given schedule is conflict-serializable.

RESULT

The schedule is **conflict-serializable**, with a possible serial order of **T3 → T1 → T2 → T4**.

QUESTION 5 — BASIC TWO-PHASE LOCKING

AIM

To demonstrate the application of **Basic Two-Phase Locking (2PL)**, identify a deadlock using a wait-for graph, and resolve the deadlock.

PROBLEM UNDERSTANDING

Consider two transactions:

T1: Read A → Write A → Read B

T2: Read B → Write B → Read A

Exclusive locks are required for write operations.

TWO PHASES OF 2PL

Growing Phase

A transaction can acquire locks but cannot release them.

Shrinking Phase

A transaction can release locks but cannot acquire new locks.

LOCK ACQUISITION

Initially:

T1: LOCK-X(A)
T2: LOCK-X(B)

Now T1 requests B:

T1 → WAIT

because T2 holds B.

T2 requests A:

T2 → WAIT

because T1 holds A.

DEADLOCK IDENTIFICATION

Current state:

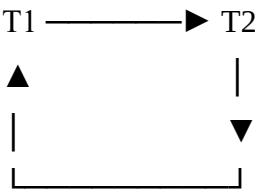
T1 holds A

T2 holds B

T1 waits for B

T2 waits for A

Wait-For Graph



This produces:

T1 → T2 → T1

Hence, a **deadlock exists**.

DEADLOCK RESOLUTION

Abort one transaction, for example:

ABORT T2

Release T2's lock:

UNLOCK(B)

T1 can now acquire B:

LOCK-X(B)

T1 completes:

COMMIT T1

2PL LOCKING SEQUENCE

T1:

LOCK-X(A)

LOCK-X(B)

READ(A)

WRITE(A)

READ(B)

WRITE(B)

COMMIT

UNLOCK(A)

UNLOCK(B)

JUSTIFICATION

Basic 2PL guarantees conflict serializability, but it does not by itself prevent deadlocks. A wait-for graph can identify circular waiting, after which a transaction can be aborted.

RESULT

Basic 2PL is successfully demonstrated, the deadlock is identified using a wait-for graph, and the deadlock is resolved by aborting one transaction.

QUESTION 6 — WAIT-DIE AND WOUND-WAIT

AIM

To apply the **Wait-Die and Wound-Wait timestamp-based deadlock prevention techniques** to transactions requesting conflicting locks.

TRANSACTION TIMESTAMPS

Consider:

T1 timestamp = 5
T2 timestamp = 10

Smaller timestamp indicates an older transaction.

Therefore:

T1 = Older
T2 = Younger

WAIT-DIE SCHEME

Rule

- Older requests a lock held by younger → **Wait**
- Younger requests a lock held by older → **Die/Abort**

Case 1

T1 requests a resource held by T2.

Since T1 is older:

T1 → WAIT

Case 2

T2 requests a resource held by T1.

Since T2 is younger:

T2 → ABORT

WOUND-WAIT SCHEME

Rule

- Older requests a lock held by younger → **Abort younger**
- Younger requests a lock held by older → **Wait**

Case 1

T1 requests a resource held by T2.

T1 is older:

T2 → ABORT
T1 → GETS RESOURCE

Case 2

T2 requests a resource held by T1.

T2 is younger:

T2 → WAIT

COMPARISON

Condition	Wait-Die	Wound-Wait
Older requests younger's lock	Wait	Abort younger
Younger requests older's lock	Abort	Wait
Deadlock prevention	Yes	Yes

JUSTIFICATION

Both schemes impose a timestamp ordering between transactions. This prevents circular waiting and therefore prevents deadlock.

RESULT

The Wait-Die and Wound-Wait schemes are successfully applied based on transaction timestamps.

QUESTION 7 — IMMEDIATE UPDATE RECOVERY

AIM

To apply the **Immediate Update recovery protocol** using transaction logs and checkpoints after a system failure.

PROBLEM UNDERSTANDING

In immediate update:

- Database changes may occur before commit.
- Log records contain old and new values.
- Committed transactions must be **REDO**.
- Uncommitted transactions must be **UNDO**.

TRANSACTION LOG

For illustration:

<START T1>
<T1, A, 100, 150>

<START T2>

<T2, B, 200, 250>

<COMMIT T1>

<CHECKPOINT>

<START T3>
<T3, C, 300, 350>

<START T4>
<T4, D, 400, 450>

<COMMIT T4>

SYSTEM FAILURE

TRANSACTION STATUS

Transaction	Status	Recovery Action
T1	Committed	REDO
T2	Uncommitted	UNDO
T3	Uncommitted	UNDO
T4	Committed	REDO

REDO OPERATIONS

T1

A: 100 → 150

Therefore:

A = 150

T4

D: 400 → 450

Therefore:

D = 450

UNDO OPERATIONS

T2

B: 200 → 250

Restore old value:

B = 200

T3

C: 300 → 350

Restore old value:

C = 300

FINAL DATABASE STATE

A = 150
B = 200
C = 300
D = 450

JUSTIFICATION

Immediate update allows changes to reach the database before commit. Therefore, recovery must redo committed transactions and undo transactions that were incomplete at the time of failure.

RESULT

The database is successfully recovered using **REDO for committed transactions and UNDO for uncommitted transactions**.

QUESTION 8 — DEFERRED UPDATE RECOVERY

AIM

To apply the **Deferred Update (NO-UNDO/REDO) recovery technique** to recover a database containing committed and uncommitted transactions.

PROBLEM UNDERSTANDING

In deferred update:

- Database modifications are postponed until the transaction commits.
- Uncommitted changes are not present in the database.
- Therefore, **UNDO is unnecessary**.
- Committed transactions are **REDONE** after failure.

TRANSACTION LOG

<START T1>
<T1, A, 100, 150>

<START T2>
<T2, B, 200, 250>

<COMMIT T1>

<START T3>

<T3, C, 300, 350>

SYSTEM FAILURE

TRANSACTION STATUS

T1 → COMMITTED

T2 → UNCOMMITTED

T3 → UNCOMMITTED

RECOVERY ANALYSIS

T1

T1 committed before failure.

Therefore:

REDO T1

A = 150

T2

T2 did not commit.

Therefore:

IGNORE T2

T3

T3 did not commit.

Therefore:

IGNORE T3

RECOVERY TABLE

Transaction	Status	Action
T1	Committed	REDO
T2	Uncommitted	Ignore
T3	Uncommitted	Ignore

FINAL STATE

A = 150

B = 200

C = 300

JUSTIFICATION

Because deferred update does not write uncommitted changes to the database, there is no need to undo incomplete transactions.

RESULT

The database is recovered using the **NO-UNDO/REDO** technique.

QUESTION 9 — STRICT 2PL FOR BANKING SYSTEM

AIM

To design a concurrency-control mechanism using **Strict Two-Phase Locking (Strict 2PL)** for a banking system where one transaction transfers funds while another transaction reads the balance.

PROBLEM UNDERSTANDING

Consider two accounts:

Account A = ₹10,000
Account B = ₹5,000

Transaction T1 transfers:

₹2,000 from A to B

Transaction T2 reads the balances simultaneously.

CONSTRAINTS

The system must ensure:

1. No dirty reads.
2. No inconsistent balance.
3. Transfer must be atomic.
4. Write locks must remain until commit.
5. T2 should not read intermediate values.

TRANSACTION T1 — FUND TRANSFER

T1:
LOCK-X(A)
LOCK-X(B)

A = A - 2000
B = B + 2000

COMMIT

UNLOCK(A)
UNLOCK(B)

After the transfer:

A = ₹8,000
B = ₹7,000

TRANSACTION T2 — BALANCE READING

T2:
LOCK-S(A)
LOCK-S(B)

READ(A)
READ(B)

COMMIT

UNLOCK(A)
UNLOCK(B)

CONCURRENT EXECUTION

Initially:

A = ₹10,000
B = ₹5,000

T1 obtains:

X-lock(A)
X-lock(B)

T1 performs the transfer:

A = ₹8,000
B = ₹7,000

T2 requests shared locks:

S-lock(A)
S-lock(B)

Since T1 still holds exclusive locks:

T2 → WAIT

T1 commits:

COMMIT T1

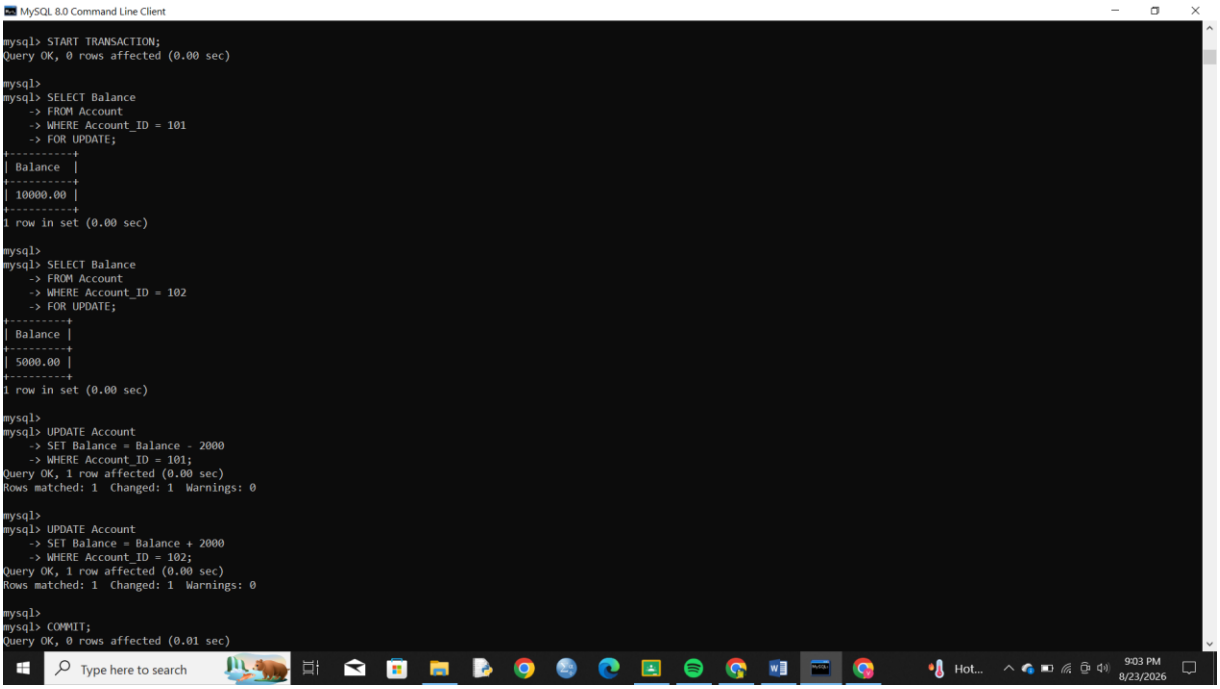
T1 releases its locks.

T2 then obtains shared locks and reads:

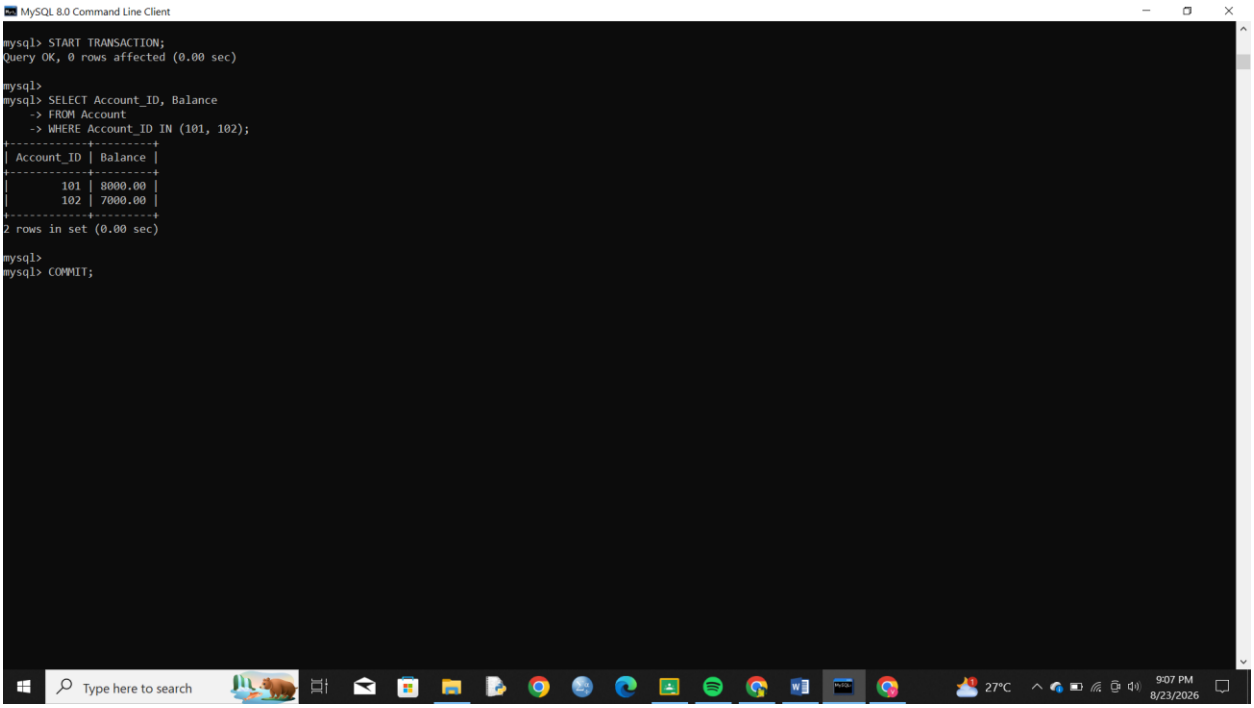
A = ₹8,000
B = ₹7,000

SQL IMPLEMENTATION

T1



T2



JUSTIFICATION

Strict 2PL holds write locks until commit. Hence T2 cannot observe the intermediate state where money has been deducted from A but not yet added to B.

RESULT

Strict 2PL successfully maintains **consistency, isolation, and atomicity** during the simultaneous banking transactions.

QUESTION 10 — HEURISTIC QUERY OPTIMIZATION PSEUDOCODE

AIM

To develop pseudocode for query optimization using **selection pushdown, early projection, and join ordering based on smaller intermediate results**.

PROBLEM UNDERSTANDING

The objective is to minimize query-processing cost by reducing:

- number of tuples,
- number of attributes,
- size of intermediate relations.

CONSTRAINT ANALYSIS

The optimizer must:

1. Push selections toward base relations.
2. Retain only required attributes.
3. Estimate intermediate relation sizes.
4. Perform smaller joins first.
5. Generate an optimized query plan.

HEURISTIC RULE 1 — SELECTION PUSH DOWN

A selection condition involving only one relation should be moved close to that relation.

Example:

σ Salary > 50000(Employee)

HEURISTIC RULE 2 — PROJECT EARLY

Only attributes needed by later operations should be retained.

For Employee:

π Emp_ID, Emp_Name, Dept_ID(Employee)

For Department:

π Dept_ID(Department)

HEURISTIC RULE 3 — ORDER JOINS

Joins producing smaller intermediate relations should be performed earlier.

PSEUDOCODE

ALGORITHM HEURISTIC_QUERY_OPTIMIZER(Query)

BEGIN

Parse Query

Construct initial relational algebra tree

FOR each selection condition

Identify the relation containing
the required attributes

IF condition involves only one relation THEN
Push selection to that relation
END IF

END FOR

FOR each relation

Identify attributes required by:

SELECT
WHERE
JOIN

Remove unnecessary attributes

Push projection toward base relation

END FOR

Estimate the size of each intermediate relation

WHILE unprocessed joins exist

Select the join that produces
the smallest intermediate relation

Perform the selected join

Recalculate intermediate result size

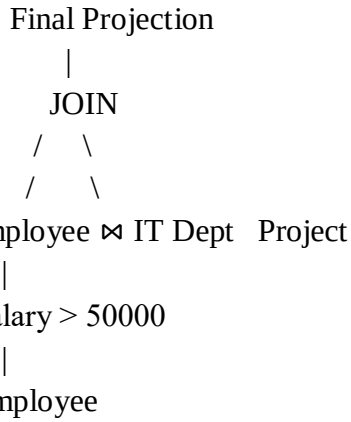
END WHILE

Generate optimized execution plan

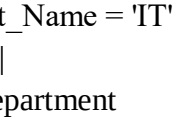
RETURN optimized plan

END

OPTIMIZED EXECUTION PLAN



IT Department:



JUSTIFICATION

Selection pushdown removes unwanted tuples early, projection pushdown reduces the number of attributes being processed, and join ordering minimizes intermediate relation sizes. These techniques reduce CPU, memory, and I/O requirements.

RESULT

A heuristic query optimizer is developed using **selection pushdown, projection pushdown, and cost-aware join ordering**.